

Rule2Test

Mỗi thay đổi quy tắc bảo hiểm trở thành test hồi quy đã được người duyệt, và bằng chứng thực thi kiểm chứng lại được.

ĐO ĐƯỢC

16.67% → 100.0%

độ phủ biên thiết kế, trước → sau

ĐO ĐƯỢC

6/6

kịch bản cổng chất lượng đúng kết luận

ĐO ĐƯỢC

286 passing

test tự động trên bản dựng này

ĐO ĐƯỢC

số đo thật trong repo ·

GIẢ ĐỊNH

giả định mô hình, có ghi căn cứ ·

TÍNH RA

tính ra từ hai loại trên

Toàn bộ số gắn nhãn "Đo được" sao chép từ artifact trong repo, do chính hệ thống chạy ra. Dữ liệu tổng hợp là dữ liệu mô phỏng, không phải đo lường môi trường thật.

Chương trình 20 phút

PHẦN	NỘI DUNG	THỜI LƯỢNG
1	Problem & Context — Bối cảnh và vấn đề	3 phút
2	Solution Overview & AI Approach — Giải pháp và cách dùng AI	4 phút
—	Live demo — nguyên mẫu chạy thật	6 phút
3	Business Case (Track 2) — ROI định lượng	4 phút
4	Differentiation / Innovation — Điểm khác biệt	2 phút
5	Feasibility & Next Steps — Khả thi và bước tiếp	1 phút

Nguyên mẫu chạy được, đóng gói thành một file .exe duy nhất, không cần cài đặt gì trên máy đích.

Một quy tắc thay đổi. Không ai chứng minh được nó làm hỏng gì.

Công ty bảo hiểm sửa điều kiện tham gia: **tuổi tối đa từ 60 lên 65**. Sau khi sửa:

- Bộ test cũ **vẫn xanh** — vì chưa ai viết test cho vùng biên vừa dịch chuyển (61–65).
- Không ai trả lời được thay đổi này **làm mất hiệu lực test nào**, và **ai** đã duyệt test mới.
- Kết quả chạy test **không truy ngược** được về điều khoản nào trong tài liệu đã sinh ra nó.
- Người duyệt bị yêu cầu tin bản tóm tắt của AI cho tài liệu họ **không kiểm lại được từng dòng**.

Vấn đề cốt lõi không phải là sinh test tự động. Vấn đề là **truy vết** và **thẩm quyền phê duyệt**.

Ba hệ quả trực tiếp

Lỗi lọt ra môi trường thật

Lỗi lệch một đơn vị ở ngưỡng tuổi hoặc ngưỡng số tiền là loại lỗi bộ test hiện có ít khi phủ nhất, và là loại gây hậu quả trực tiếp lên khách hàng.

Công sức lặp lại thủ công

Mỗi lần đổi quy tắc, QA phải đọc lại tài liệu, rà bộ test cũ, thiết kế case biên và soạn hồ sơ truy vết — gần như làm lại từ đầu.

Không có bằng chứng cho kiểm toán

Khi bị hỏi "vì sao tin rằng thay đổi này an toàn", câu trả lời hiện nay là một ảnh chụp màn hình bộ test xanh — không gắn với điều khoản, không gắn với người duyệt.

Năm chặng, một chuỗi truy vết liên tục

CHẶNG	VIỆC XẢY RA	AI QUYẾT ĐỊNH
Tiếp nhận	Hai phiên bản quy tắc; lưu nguyên bytes gốc kèm mã băm	Máy
Phân tích	Delta quy tắc, ảnh hưởng, tìm lỗ hổng có chặn, sinh test đề xuất	Máy
Duyệt	Phê duyệt tường minh từng test, gắn với revision	Con người
Thực thi	Chỉ chạy test đã duyệt, đối chiếu với hệ thống độc lập	Máy
Chứng minh	Gói bằng chứng + cổng chất lượng GO / NO-GO	Máy

Mỗi chặng gắn với một số revision. Gửi revision cũ thì hệ thống từ chối chứ không ghi đè âm thầm. Mở lại phiên duyệt sẽ xoá phê duyệt cũ và làm mất hiệu lực kết luận GO trước đó.

AI đề xuất. Engine quyết định. Con người phê duyệt.

AI được làm

- Đọc tài liệu văn xuôi, rút ra quy tắc có cấu trúc
- Đề xuất *đầu vào* test từ kho test đã duyệt
- Tạo vector cho tìm kiếm lại

AI không bao giờ được làm

- Tính kết quả **mong đợi** — oracle kiểu tĩnh làm việc này
- Quyết định PASS / FAIL
- Tự phê duyệt đề xuất của chính nó
- Nhìn thấy đáp án trước khi trả lời

Bốn rào chắn kỹ thuật

- **Trích dẫn đúng nguyên văn.** Mỗi quy tắc AI rút ra phải kèm đúng dòng gốc. Sai một ký tự là bị loại.
- **Không tự thử lại, không âm thầm quay về mock.** Lỗi provider được phân loại thành 11 loại kèm hướng khắc phục.
- **Ghim mã digest của model.** Sai model là dừng ngay lúc khởi động.
- **Hệ thống được test không bao giờ nhận kết quả mong đợi hay bản quy tắc.**

Ranh giới tin cậy

Tài liệu (không tin cậy) → **AI đề xuất** → kiểm tra schema + trích dẫn đúng dòng → **người duyệt quy tắc** → phân tích & oracle tính mong đợi → **người duyệt test** → hệ thống độc lập tính thực tế → bằng chứng → cổng GO/NO-GO

Không có mũi tên nào đi ngược. Không đường nào chạm tới bước thực thi mà không qua phê duyệt của con người. Lỗi provider dừng lại ở bước kiểm tra, được phân loại, và **không** được thay bằng kết quả giả lập.

ĐO ĐƯỢC

5/10

trích xuất — AI giả lập (chỉ phát lại fixture)

ĐO ĐƯỢC

4/10

trích xuất — model thật, 10 case tiếng Nhật tự soạn

ĐO ĐƯỢC

9/9

truy hồi top-1 trên kho đã duyệt

Nói thẳng: số của AI giả lập cao hơn model thật, nhưng giả lập chỉ phát lại đúng mẫu có sẵn. Không được dùng nó để chứng minh năng lực AI. Con số thật hiện là 4/10 và còn thấp.

Nguyên mẫu chạy thật

Bạn sắp thấy, theo đúng thứ tự:

- Tạo workflow từ thay đổi tuổi 60 → 65, lưu nguyên tài liệu gốc kèm mã băm.
- Phân tích: **14 test đề xuất**, chỉ ra vùng biên chưa ai phủ.
- Phê duyệt tường minh — không có gì tự duyệt.
- Chạy sạch: **14 pass / 0 fail** → cổng **GO**.
- **Bơm lỗi lệch một đơn vị ở đúng ngưỡng** → **13 pass / 1 fail** → cổng **NO-GO**, phản ví dụ là một con số tuổi cụ thể, không phải stack trace.
- Tải gói bằng chứng, mở bảng chẩn đoán vận hành.

Chạy từ một file .exe duy nhất, không cần cài Python hay bất cứ thứ gì trên máy đích. Dữ liệu ghi cạnh file .exe nên còn nguyên sau khi tắt và mở lại. Nếu demo trực tiếp không chạy được, có sẵn bản phát lại ảnh chụp ngoại tuyến kèm đúng lời dẫn này.

Cách chúng tôi tính ROI — và cách kiểm tra lại

Dự án này **chưa đo** thời gian tiết kiệm thực tế. Vì vậy không có con số ROI nào ở đây được trình bày như số đo. Thay vào đó là một mô hình công khai.

ĐO ĐƯỢC

số đo thật trong repo ·

GIẢ ĐỊNH

giả định mô hình, có ghi căn cứ ·

TÍNH RA

tính ra từ hai loại trên

Ba cam kết về số liệu

- Mọi đầu vào đều ghi rõ **căn cứ** ngay trên slide sau.
- Mọi phép tính đều **hiện công thức**.
- Trình bày **độ nhạy** thay vì bảo vệ một con số duy nhất.

Phạm vi cố tình chọn nhỏ

Một team QA bảo hiểm — 8 người. Đây là đơn vị nhỏ nhất có thể kiểm chứng. Phần nhân rộng lên BU và tập đoàn được trình bày riêng, tách khỏi con số gốc.

Nếu quý vị cho rằng một giả định của chúng tôi sai, bảng độ nhạy ở slide sau chỉ ra ngay kết quả sẽ thành bao nhiêu — thay vì phải tin hay không tin một con số.

Bảng giả định của mô hình

ĐẦU VÀO	GIÁ TRỊ	CĂN CỨ
Quy mô team	8 người	Quy mô một team QA sản phẩm bảo hiểm điển hình. Cố tình chọn đơn vị nhỏ nhất để con số dễ bảo vệ.
Số lần đổi quy tắc	36 lần/năm	≈3 lần/tháng: thay đổi tham số sản phẩm, quy định pháp lý theo quý, quy tắc chiến dịch.
Công sức hiện tại, mỗi lần đổi	16 giờ/lần	2 người-ngày: đọc và so tài liệu ~5h, rà bộ test bị ảnh hưởng và thiết kế case mới ~7h, soạn hồ sơ truy vết ~4h.
Công sức khi có công cụ	6 giờ/lần	Phân tích delta, tìm lỗi hổng, sinh test và đóng gói bằng chứng do máy làm. Phần còn lại là DUYỆT — việc này không tự động hoá và không được tính là tiết kiệm.
Tỷ lệ quy tắc DSL diễn đạt được	0,50 tỷ lệ	DSL hiện hỗ trợ 2 trường và 8 toán tử. Quy tắc nằm ngoài đó bị hệ thống TỪ CHỐI chứ không đoán, nên chỉ một nửa số lần đổi được tính vào ROI.
Chi phí đầy đủ một giờ QA	300.000 VND/giờ	Lương, bảo hiểm, quản lý, thiết bị ≈45 triệu/tháng ÷ ~147 giờ làm việc/tháng.
Giờ làm việc hữu ích	1.760 giờ/người/năm	220 ngày làm việc × 8 giờ. Dùng làm mẫu số cho cách tính năng suất thứ hai.

Hệ số phủ DSL là giả định *thận trọng nhất*: hệ thống **từ chối** quy tắc nó không diễn đạt được thay vì đoán, nên chỉ một nửa số lần đổi quy tắc được tính vào ROI.

Giờ tiết kiệm và giá trị quy đổi

Giờ tiết kiệm/năm = số lần đổi quy tắc × hệ số phủ DSL × (giờ thủ công – giờ có công cụ)

= 36 × 0,50 × (16 – 6) = **180 giờ/năm**

TÍNH RA 180 giờ tiết kiệm mỗi năm, một team 8 người	TÍNH RA 31,2% tăng năng suất trên khối lượng việc hồi quy do đổi quy tắc	TÍNH RA 1,28% trên tổng năng lực cả team — mẫu số rộng hơn	TÍNH RA 54 triệu VND giá trị quy đổi mỗi năm
--	---	---	---

Vì sao trình bày hai tỷ lệ năng suất? Công cụ chỉ tác động vào phần việc hồi quy do đổi quy tắc (576 giờ/năm), không phải toàn bộ công việc của team (14.080 giờ/năm). Nêu cả hai để con số không bị hiểu sai theo hướng có lợi cho chúng tôi.

Thời gian **phê duyệt của con người không được tính là tiết kiệm**. Việc đó không tự động hoá và không nên tự động hoá — nó là thẩm quyền, không phải chi phí cần cắt.

Độ nhảy — con số đổi thế nào khi giả định đổi

KỊCH BẢN	PHỦ DSL	LẦN ĐỔI/NĂM	GIỜ/NĂM	VND/NĂM	
Hiện tại (giai đoạn 12)		50%	36	180	54 triệu VND
Mở rộng DSL (giai đoạn 16)		80%	36	288	86 triệu VND
Tần suất đổi quy tắc cao		50%	60	300	90 triệu VND
Cả hai		80%	60	480	144 triệu VND

Biến nhảy nhất là **hệ số phủ DSL**. Nâng nó là mục tiêu của giai đoạn 16 trong lộ trình (mở rộng số trường và toán tử), và đó cũng là hạng mục kỹ thuật có đòn bẩy ROI cao nhất.

Bảng này thay cho một con số đơn lẻ. Nếu quý vị cho rằng giả định của chúng tôi sai, bảng chỉ ra ngay kết quả sẽ thành bao nhiêu — thay vì phải tin hay không tin một con số duy nhất.

Khả năng nhân rộng và tái sử dụng

PHẠM VI	SỐ QA	GIỜ/NĂM	VND/NĂM	
1 team QA		8	180	54 triệu VND
5 team / 1 Business Unit		40	900	270 triệu VND
20 team / nhiều BU		160	3.600	1.080 triệu VND

Vì sao nhân rộng được — cơ chế, không phải lời hứa

- **Engine không gắn nghiệp vụ cụ thể.** Quy tắc là *dữ liệu*, không phải mã nguồn.
- **Kho tri thức dùng chung.** Test đã duyệt tái sử dụng được; kết quả mong đợi luôn *tính lại*.
- **Không phụ thuộc hạ tầng.** Lỗi chỉ dùng thư viện chuẩn; triển khai đơn vị mới là chép một file.

Bảng trên là phép nhân tuyến tính từ mô hình một team, chưa trừ chi phí triển khai, đào tạo và vận hành khi mở rộng.

Giá trị tránh rủi ro — trình bày riêng, có lý do

Giá trị lớn nhất của hệ thống này nhiều khả năng **không** nằm ở giờ công tiết kiệm, mà ở **lỗi không lọt ra môi trường thật**. Chúng tôi tách riêng phần này vì không đo được chi phí một lỗi lọt trong bối cảnh của quý vị.

Cơ chế — đã đo

ĐO ĐƯỢC

Độ phủ biên thiết kế: **16.67% → 100.0%**. Mutation score: **100.0%** trên cùng kịch bản. Cồng chất lượng: **6/6** kịch bản đúng kết luận, kể cả các ca cố tình phải NO-GO.

Giá trị — chưa đo

GIẢ ĐỊNH

Phụ thuộc quy mô hợp đồng, mức bồi hoàn và rủi ro pháp lý. Đây là con số **của quý vị**.

Chúng tôi chứng minh được phần **cơ chế**: lỗi lệch một đơn vị ở đúng ngưỡng bị bắt, và cồng chuyển sang NO-GO. Quý vị nhân nó với chi phí một lỗi lọt trong tổ chức mình.

Chúng tôi chủ động không nhân sẵn con số đó để trình bày như ROI. Làm vậy sẽ biến một giả định thành một lời khẳng định.

Khác gì so với các cách làm hiện có

CÁCH TIẾP CẬN	SINH TEST	AI TÍNH KẾT QUẢ MONG ĐỢI	TRUY VẾT VỀ ĐIỀU KHOẢN	CHẶN PHÁT HÀNH
Viết test thủ công	Người	Người	Không có	Không
Hỏi LLM sinh test	AI	AI — không kiểm chứng được	Không có	Không
Công cụ sinh test theo mô hình	Máy	Máy	Không gắn tài liệu	Không
Rule2Test	Máy + AI đề xuất	Oracle kiểu tĩnh	Trích dẫn đúng dòng + bấm	Cổng GO/NO-GO

Ba điểm mới thực sự

- **Trích dẫn nguyên văn là điều kiện bắt buộc.** AI không chỉ được dòng gốc thì output bị loại — cách duy nhất để người duyệt kiểm lại trong thời gian hữu hạn.
- **Hai bên tính độc lập.** Oracle tính mong đợi, hệ thống riêng tính thực tế, không chia sẻ mã.
- **Bảng chứng gắn revision.** Sửa test là phê duyệt hết hiệu lực; mở lại phiên duyệt là GO mất hiệu lực.

Bảng chứng cho từng điểm khác biệt

KHẲNG ĐỊNH	KIỂM CHỨNG BẰNG	SỐ ĐO
Bắt được lỗi lệch ngưỡng	Bơm lỗi biên vào hệ thống độc lập, chạy thật	13/14 pass → NO-GO
Tìm ra vùng biên chưa phủ	Độ phủ biên trước và sau khi sinh test	16.67% → 100.0%
Bộ test đủ mạnh	Mutation — cố tình làm sai quy tắc	100.0%
Cổng chất lượng đúng hợp đồng	6 kịch bản độc lập, gồm cả ca phải NO-GO	6/6
AI không tự duyệt	Test tự động khẳng định không có phê duyệt tự động	có test
Không tự thử lại khi lỗi	Provider giả lập đếm số lần gọi	gọi đúng 1 lần
Log không rò dữ liệu	Danh sách trường cho phép, test khẳng định	có test

Tất cả đều nằm trong bộ 286 passing test tự động, chạy lại được bằng một lệnh. Mọi số đều truy về được artifact trong repo.

Đã sẵn sàng tới đâu

Đã có, chạy được

- 12 giai đoạn hoàn chỉnh, từ hợp đồng miền tới cổng chất lượng
- Nguyên mẫu đóng gói một file .exe, không cần cài gì
- 286 passing test tự động, chạy ổn định
- Log có cấu trúc, trace, metrics, phân loại lỗi provider
- Bộ tài liệu và dữ liệu demo mang đi được

Chưa có — nói thẳng

- Chưa xác thực, chưa phân quyền; tên người duyệt là tự khai
- Chưa có nhãn do chuyên gia bảo hiểm gán cho bộ đánh giá AI
- DSL còn nhỏ: 2 trường, 8 toán tử
- Băm là kiểm tra toàn vẹn, chưa phải chữ ký số
- Chưa nối hệ thống thật, chưa chạy trong CI

Rào cản lớn nhất để triển khai thật không phải kỹ thuật lõi — mà là dữ liệu có nhãn chuyên gia và việc nối vào hệ thống thật. Cả hai đều nằm ngay trong hai giai đoạn kế tiếp.

Bốn giai đoạn tiếp theo

GĐ	MỤC TIÊU	KẾT QUẢ ĐO ĐƯỢC
13	Bộ đánh giá quy mô lớn, có nhãn chuyên gia, tách tập kiểm định giữ riêng	Thay "4/10 trên 10 case tự soạn" bằng con số trên hàng trăm case có nhãn
14	Nối hệ thống thật qua adapter HTTP sẵn có; chạy trong CI, chặn merge khi NO-GO	Số lần chặn được thay đổi rủi ro trước khi vào môi trường thật
15	Xác thực, phân quyền, bằng chứng ký số, lưu trữ chỉ-ghi-thêm	Bằng chứng đủ điều kiện dùng cho kiểm toán
16	Mở rộng DSL, test biến hình	Hệ số phủ DSL 50% → 80%: ROI tăng từ 54 triệu VND lên 86 triệu VND/năm cho một team

Giai đoạn 16 là hạng mục có đòn bẩy ROI cao nhất, và bảng độ nhạy ở phần Business Case đã lượng hoá đúng mức đòn bẩy đó.

Những gì chúng tôi *không* khẳng định

- **Độ phủ không phải tỷ lệ pass.** Test fail vẫn tính là đã chạy qua đầu vào đó.
- **GO không phải giấy phép phát hành.** Đó là đánh giá hồi quy cục bộ cho đúng một revision.
- **Số của AI giả lập không chứng minh năng lực AI.** Giả lập chỉ phát lại fixture. Model thật đạt 4/10.
- **Prompt v2 được sửa sau khi xem lỗi của v1** → không phải kiểm định độc lập. Chúng tôi ghi điều này trong tài liệu thay vì giấu đi.
- **Mọi con số ROI trong phần Business Case là mô hình, không phải đo lường.** Giả định và công thức đều hiện trên slide để quý vị kiểm lại.
- **Chưa đo:** mức chấp nhận của người duyệt thật, độ chính xác môi trường thật, chi phí token.

Chúng tôi chọn để quý vị tin vào năm con số kiểm chứng được, thay vì nghi ngờ năm mươi con số không kiểm chứng được.

Mọi số "Đo được" truy về artifact trong repo: docs/BENCHMARKS.md. Mọi giả định mô hình nằm trong một khối duy nhất của scripts/build_final_deck.py — sửa một chỗ, cả deck cập nhật.